[166] M. Dibak, L. Klein, A. Krämer, and F. Noé, Temperature steerable flows and boltzmann generators, Phys. Rev. Res. **4**, L042005 (2022).

[167] M. Invernizzi, A. Krämer, C. Clementi, and F. Noé, Skipping the replica exchange ladder with normalizing flows, The Journal of Physical Chemistry Letters **13**, 11643 (2022).

[168] M. Xu, L. Yu, Y. Song, C. Shi, S. Ermon, and J. Tang, Geodiff: A geometric diffusion model for molecular conformation generation, arXiv preprint arXiv:2203.02923 (2022).

[169] A. Coretti, S. Falkner, P. Geissler, and C. Dellago, Learning mappings between equilibrium states of liquid systems using normalizing flows, arXiv preprint arXiv:2208.10420 (2022).

[170] S. van Leeuwen, A. P. de Alba Ortíz, and M. Dijkstra, A boltzmann generator for the isobaric-isothermal ensemble (2023), arXiv:2305.08483.

[171] M. Gabrié, G. M. Rotskoff, and E. Vanden-Eijnden, Adaptive monte carlo augmented with normalizing flows, Proceedings of the National Academy of Sciences **119**, e2109420119 (2022).

[172] G. Jung, G. Biroli, and L. Berthier, Normalizing flows as an enhanced sampling method for atomistic supercooled liquids, Machine Learning: Science and Technology **5**, 035053 (2024).

[173] B. McNaughton, M. Milošević, A. Perali, and S. Pilati, Boosting monte carlo simulations of spin glasses using autoregressive neural networks, Phys. Rev. E **101**, 053312 (2020).

[174] M. Hibat-Allah, E. M. Inack, R. Wiersema, R. G. Melko, and J. Carrasquilla, Variational neural annealing, Nature Machine Intelligence **3**, 952 (2021).

[175] D. Wu, R. Rossi, and G. Carleo, Unbiased monte carlo cluster updates with autoregressive neural networks, Phys. Rev. Res. **3**, L042024 (2021).

[176] E. M. Inack, S. Morawetz, and R. G. Melko, Neural annealing and visualization of autoregressive neural networks in the newman–moore model, Condensed Matter **7**, 38 (2022).

[177] S. Ciarella, J. Trinquier, M. Weigt, and F. Zamponi, Machine-learning-assisted monte carlo fails at sampling computationally hard problems, Machine Learning: Science and Technology **4**, 010501 (2023).

[178] M. J. Schuetz, J. K. Brubaker, Z. Zhu, and H. G. Katzgraber, Graph coloring with physics-inspired graph neural networks, Phys. Rev. Res. **4**, 043131 (2022).

[179] M. J. Schuetz, J. K. Brubaker, and H. G. Katzgraber, Combinatorial optimization with physics-inspired graph neural networks, Nature Machine Intelligence **4**, 367 (2022).

[180] M. S. Albergo, G. Kanwar, and P. E. Shanahan, Flow-based generative models for markov chain monte carlo in lattice field theory, Phys. Rev. D **100**, 034515 (2019).

[181] G. Kanwar, M. S. Albergo, D. Boyda, K. Cranmer, D. C. Hackett, S. Racaniere, D. J. Rezende, and P. E. Shanahan, Equivariant flow-based sampling for lattice gauge theory, Phys. Rev. Lett. **125**, 121601 (2020).

[182] P. de Haan, C. Rainone, M. C. Cheng, and R. Bondesan, Scaling up machine learning for quantum field theory with equivariant continuous flows, arXiv preprint arXiv:2110.02673 (2021).

[183] M. Gerdes, P. de Haan, C. Rainone, R. Bondesan, and M. C. Cheng, Learning lattice quantum field theories with equivariant continuous flows, SciPost Physics **15**, 238 (2023).

[184] D. Luo, G. Carleo, B. K. Clark, and J. Stokes, Gauge equivariant neural networks for quantum lattice gauge theories, Phys. Rev. Lett. **127**, 276402 (2021).

[185] T. Marchand, M. Ozawa, G. Biroli, and S. Mallat, Wavelet conditional renormalization group, arXiv preprint arXiv:2207.04941 (2022).

[186] M. C. Angelini and F. Ricci-Tersenghi, Modern graph neural networks do worse than classical greedy algorithms in solving combinatorial optimization problems like maximum independent set, Nature Machine Intelligence **5**, 29 (2023).

[187] S. Boettcher, Inability of a graph neural network heuristic to outperform greedy algorithms in solving combinatorial optimization problems, Nature Machine Intelligence **5**, 24 (2023).

[188] S. Boettcher, Deep reinforced learning heuristic tested on spin-glass ground states: The larger picture, Nature Communications **14**, 5658 (2023).

[189] D. Ghio, Y. Dandi, F. Krzakala, and L. Zdeborová, Sampling with flows, diffusion and autoregressive neural networks: A spin-glass perspective, arXiv preprint arXiv:2308.14085 (2023).

[190] Links to Workshop and YouTube channel.

[191] L. Berthier, E. Flenner, C. J. Fullerton, C. Scalliet, and M. Singh, Efficient swap algorithms for molecular dynamics simulations of equilibrium supercooled liquids, Journal of Statistical Mechanics: Theory and Experiment **2019**, 064004 (2019).

[192] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, C. Gulcehre, F. Song, A. Ballard, J. Gilmer, G. Dahl, A. Vaswani, K. Allen, C. Nash, V. Langston, C. Dyer, N. Heess, D. Wierstra, P. Kohli, M. Botvinick, O. Vinyals, Y. Li, and R. Pascanu, Relational inductive biases, deep learning, and graph networks (2018), arXiv:1806.01261.

[193] J. Paret and D. Coslovich, partycls: A python package for structural clustering, JOSS **6**, 3723 (2021).

[194] P. J. Steinhardt, D. R. Nelson, and M. Ronchetti, Bond-orientational order in liquids and glasses, Phys. Rev. B **28**, 784 (1983).

[195] D. Coslovich, M. Ozawa, and W. Kob, Dynamic and thermodynamic crossover scenarios in the kob-andersen mixture: Insights from multi-cpu and multi-gpu simulations, The European Physical Journal E **41**, 1 (2018).

Supplemental material: Roadmap on machine learning glassy dynamics

arXiv:2311.14752v2 [cond-mat.soft] 26 Sep 2024

In this supplemental material (SM) we provide the technical details necessary to reproduce the data shown in the benchmarking section V of the main manuscript. Additionally, we present further benchmarking.

I. SYSTEMS AND METHODS

A. KA system (3D)

Most benchmarks were performed for the Kob-Andersen binary mixture (KA) in three dimensions [S1]. The basic potential used in KA is the Lennard-Jones potential,

$$V_{\alpha\beta}(r_{ij}) = \begin{cases} 4\epsilon_{\alpha\beta}\left[\left(\frac{\sigma_{\alpha\beta}}{r_{ij}}\right)^{12} - \left(\frac{\sigma_{\alpha\beta}}{r_{ij}}\right)^{6}\right] & r_{ij} < r^{\text{cut}}_{\alpha\beta} \\ 0 & \text{otherwise,} \end{cases}$$

where $r_{ij} = |\boldsymbol{r}_i - \boldsymbol{r}_j|$ is the distance between particles i and j, of type α and β, respectively. The KA mixture is characterized by its non-additive interactions, $\epsilon_{11} = 1.0$, $\epsilon_{12} = 1.5$, $\epsilon_{22} = 0.5$ and $\sigma_{11} = 1.0$, $\sigma_{12} = 0.8$, $\sigma_{22} = 0.88$, between the two particles types. The cutoff is $r^{\text{cut}}_{\alpha\beta} = 2.5\sigma_{\alpha\beta}$.

All details on the specific KA mixture and the dataset used in this roadmap are given in Ref. [S2]. The system, for example, includes additional contributions in the potential to make it continuous up to the second derivative. The relaxation dynamics of each structure has been simulated using $N_R = 32$ independent replicas to calculate the isoconfigurational average. All results have been extracted by solely considering type 1 particles. The networks have been trained on $N_S = 400$ structures, with $N_1 = 3277$ type 1 particles each, resulting in a total of $N_d \approx 10^6$ data points. The testset consists of $N_S = 100$ independent structures.

B. KA2D system

The two-dimensional KA2D system is a variation of the above described KA mixture. In addition to the change of dimensionality, KA2D consists of three particle types, instead of two. This change enables to apply the swap Monte Carlo algorithm [S3, S4] and thus go to very low temperatures [S5]. All details on the KA2D system are provided in the main text and the SM of Ref. [S5].

The isoconfigurational average has been calculated over $N_R = 20$ independent replicas, and $N_S = 1500$ structures are provided for training. Each structure consists of 600 type 1 particles, which thus also yields a total of $N_d \approx 10^6$ data points for training.

For both the KA and the KA2D system, the structural relaxation time τ_α is defined as,

$$F_s(\tau_\alpha) = e^{-1}, \tag{S1}$$

where $F_s(t) = \langle \sum_i \cos(q\Delta r_i(t))\rangle$ is the incoherent intermediate scattering function with $q \approx 2\pi/\sigma$ and $\Delta r_i(t) = |\boldsymbol{r}_i(t) - \boldsymbol{r}_i(0)|$. The structural relaxation time τ_α therefore describes the time scale on which the particles move on average approximately one particle diameter σ.

Equilibrated structures and labels calculated from the relaxation dynamics for both KA and KA2D will be uploaded in combination with this roadmap for future research (see 10.5281/zenodo.10118191).

C. Propensity (labels)

The input of each ML method are the positions of each particle in a given structure or derived structural descriptors. The labels that are used for training and testing are dynamical quantities characterizing structural relaxation at various time scales t. As introduced in Sec. III of the main text we use propensities to remove any effects of the initial velocities.

For each figure in the main text, except Fig. 5, the results are based on the propensities of displacement, $\mathcal{R}_i = \langle \Delta r_i(t)\rangle_{\text{iso}} = N_R^{-1}\sum_{k=1}^{N_R}\left|\boldsymbol{r}_i^k(t) - \boldsymbol{r}_i(0)\right|$, where $\boldsymbol{r}_i^k(t)$ denotes the position of particle i in replica k. In Fig. 5 we have reported results for the bond-breaking propensity of particle i, $\mathcal{C}_B^i(t) = \langle n_t^i/n_0^i\rangle_{\text{iso}}$. Here, n_t^i describes the number of original nearest neighbours particle i still has after a time t and n_0^i is the initial number of neighbours [S6]. The quantity therefore decays from $\mathcal{C}_B^i(t=0) = 1$ to $\mathcal{C}_B^i(t\to\infty) = 0$. Details and parameters are described in the SM of Ref. [S5].

D. BOTAN

The model architecture of BOTAN [S2] is based on that of the original GNN proposed by Bapst *et al.* [S7]. The difference is that BOTAN is equipped with an additional MLP layer as a decoder for targets on graph edges. BOTAN is implemented as an "interaction network" [S8] consisting of a pair of two-layer MLPs with the size of $(64, 64)$. Each of the MLPs is connected to nodes and edges, respectively, and performs message passing with the other.

The input graphs are constructed by choosing each particle as a node of the graph, and "connecting" nearest-neighbour pairs by edges. The threshold distance for the nearest neighbours are set to $2.0\sigma_{11}$ (both for KA and 2DKA). For recursive message passing, the MLPs repeat the message passing iteratively 7 (KA) or 8 (2DKA)

times, but the results are not strongly affected by slight changes in this number. The target quantity on the edges is, similarly to Ref. [S2], the isoconfigurational average of distance change between particle pairs which are connected via the edges. The loss function is defined as $\mathcal{L} = 0.4\mathcal{L}_{\mathcal{N}} + 0.6\mathcal{L}_{\mathcal{E}}$, where $\mathcal{L}_{\mathcal{N}}$ and $\mathcal{L}_{\mathcal{E}}$ denote the MSE losses for node target (particle propensity) and edge target (propensity for pair-distance change), respectively. Details and visualisation of the method are presented in Ref. [S2].

The BOTAN GNN model contains about 54,200 weight parameters (both for 2D and 3D). The training is started by using pretrained model parameters (optimized to yield the edge target quantities in advance), and is performed over 1,000 epochs on 400 snapshots (KA) and over 2,000 epochs on 1,500 snapshots (2DKA), respectively. The batch size is fixed to one graph. The training has been conducted separately for each temperature and time, by using the Adam optimizer with learning rates of 10^{-4} (KA) and 2×10^{-4} (2DKA). The overall training time amounted to 4 hours (KA) and 11.5 hours (2DKA) on an NVIDIA A100 Tensor Core GPU (40GB SXM) on Aquarius subsystem of Wisteria/BDEC-01 Supercomputer at Information Technology Center, University of Tokyo, with the host CPUs being 2 Intel Xeon Platinum 8360Y. These training times do not include the elapsed time of the data loader. The code and pretrained model parameters for 3D are provided on a GitHub repository (`https://github.com/h3-Open-BDEC/pyg_botan`).

E. CAGE

As described in Ref. [S9], we train a simple Ridge regression model to fit the dynamics, based on the structure of both the initial state and the cage state. The cage state is defined as the average position of particles before any rearrangement occurs. To obtain the cage state, we perform a Monte Carlo simulation where all the particles are confined to a sphere of radius r_c^α. For both glassy systems we use $r_c^\alpha = 1.25\sigma_{\alpha\alpha}$ with $\sigma_{\alpha\alpha}$ the particle diameter of type α.

To capture the local structure, we use recursive, rotationally invariant parameters that capture both the radial density, as well as the n-fold symmetry in various shells around each particle. For the 3D system, we use exactly the same set of descriptors as in Ref. [S9]. For the 2D system we use the same density parameters as for 3D [S9], however the angular parameters are altered to better reflect 2D symmetry. Specifically, to capture the n-fold symmetry for the 2D system we use

$$\Phi_i^l(r,\delta) = \sqrt{\phi_i^l \cdot (\phi_i^l)^*}$$

with

$$\phi_i^l(r,\delta) = \frac{1}{Z}\sum_{i\neq j} e^{\frac{-(r_{ij}-r)^2}{2\delta^2}} e^{il\theta_{ij}},$$

and

$$Z = \sum_{i\neq j} e^{\frac{-(r_{ij}-r)^2}{2\delta^2}},$$

where θ_{ij} is the angle between a fixed axis (e.g., x- or y-axis) and the bond joining the ith particle with a particle j. In both 2D and 3D we consider $l \in \{1, 12\}$.

Combining both the radial density and the angular parameters, we obtain a total of 462 parameters for the KA2D system:

- 294 parameters that described the radial density up to the 5th minimum in the pair correlation function (which is located at $4.8\sigma_{11}$).
  - 60 equally spaced in the interval $r/\sigma_{11} \in (0.5, 2.0]$ with $\delta = 0.025$.
  - 20 equally spaced in the interval $r/\sigma_{11} \in (2.0, 3.0]$ with $\delta = 0.050$.
  - 18 equally spaced in the interval $r/\sigma_{11} \in (3.0, 4.8]$ with $\delta = 0.100$.

  Note that the radial density functions are type specific, such that in the ternary system the total number of functions above is multiplied by three.

- 168 parameters that capture the n-fold symmetry up to the second minimum of the pair correlation function (located at $2.3\sigma_{11}$).
  - 14 equally spaced in the interval $r/\sigma_{11} \in [1.0, 2.3]$ with $\delta = 0.1$ and $l \in [1, 12]$.

  Note that the radial functions do not take the particle species into account.

As described in Ref [S9], for the 3D system we obtain a total of 366 parameters. In addition to the zeroth order generation parameters, for both the 2D and 3D system, we additionally include two generations of structural parameters that are iteratively averaged over the nearest neighbours (see Ref. [S10]).

The dataset on which we train our Ridge regression thus includes three generations of structural descriptors for both the initial structure, as well as the cage state. Additionally, for each particle we include the distance between the cage state and the initial positions $\Delta r_i^{\text{cage}} = |\boldsymbol{r}_i^{\text{cage}} - \boldsymbol{r}_i^{\text{init}}|$. This means that the local structure of each particle in the 2D system is described by a total of 2197 parameters, and in the 3D system a total of 2773 parameters.

To predict the dynamics, we standardise the data and then train the Ridge regression model on 300 (KA2D) and 100 (3D KA) snapshots, respectively. The only free parameter that is tuned for this ML method is the regularization parameter α, which sets the strength of the penalty for large weights in the Ridge regression. We